VTU28804

WEEK-1

SESSION-1:

Task-1:Java Lambda Expressions

//code

```
public PerformOperation isOdd() {
        return (int a) -> a % 2 != 0;
    }


    public PerformOperation isPrime() {
        return (int a) -> {
            if (a < 2) return false;
            for (int i = 2; i <= Math.sqrt(a); i++) {
                if (a % i == 0) return false;
            }
            return true;
        };
    }
    public PerformOperation isPalindrome() {
        return (int a) -> {
            String original = Integer.toString(a);
            String reversed = new
StringBuilder(original).reverse().toString();
            return original.equals(reversed);
```

```
        };
    }
```

Task-2:Java Comparator

```
import java.util.*;
class Checker implements Comparator<Player>{
  @Override//Written part
    public int compare(Player a,Player b){
        return Comparator.comparingInt((Player p) -> p.score).reversed()
                    .thenComparing(p -> p.name)
                    .compare(a, b);
}
}
class Player{
    String name;
    int score;

    Player(String name, int score){
        this.name = name;
        this.score = score;
    }
}

class Solution {
```

```
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();

        Player[] player = new Player[n];
        Checker checker = new Checker();

        for(int i = 0; i < n; i++){
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();

        Arrays.sort(player, checker);
        for(int i = 0; i < player.length; i++){
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}
```

SESSION-2

Task-1:Java Sort

```
class Student{
	private int id;
	private String fname;
	private double cgpa;
	public Student(int id, String fname, double cgpa) {
		super();
		this.id = id;
		this.fname = fname;
		this.cgpa = cgpa;
	}
	public int getId() {
		return id;
	}
	public String getFname() {
		return fname;
	}
	public double getCgpa() {
		return cgpa;
	}
}
```

```
public class Solution
{
	public static void main(String[] args){
		Scanner in = new Scanner(System.in);
		int testCases = Integer.parseInt(in.nextLine());

		List<Student> studentList = new ArrayList<Student>();
		while(testCases>0){
			int id = in.nextInt();
			String fname = in.next();
			double cgpa = in.nextDouble();

			Student st = new Student(id, fname, cgpa);
			studentList.add(st);

			testCases--;
		}

studentList.sort(Comparator.comparing(Student::getCgpa).reversed()

.thenComparing(Student::getFname).thenComparingInt(Student::getId));
	for(Student st: studentList){
			System.out.println(st.getFname());}}}
```

Session-3

Task-1:Sorting: Comparator

```
import java.util.*;
class Player {
        String name;
        int score;
        Player(String name, int score) {
                this.name = name;
                this.score = score;}}
class Checker implements Comparator<Player> {
        public int compare(Player a, Player b) {
      if (a.score == b.score) {
         return a.name.compareTo(b.name);
       } else {
         return Integer.compare(b.score, a.score);  }}}
public class Solution {
   public static void main(String[] args) {
       Scanner scan = new Scanner(System.in);
       int n = scan.nextInt();
       Player[] player = new Player[n];
       Checker checker = new Checker();
       for(int i = 0; i < n; i++){
          player[i] = new Player(scan.next(), scan.nextInt()); }
```

```
        scan.close();


        Arrays.sort(player, checker);
        for(int i = 0; i < player.length; i++){
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}
```

SESSION-4

Task-1:**Running Sum of 1D Array**

```
class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] += nums[i - 1];
        }
        return nums;
    }
    public static void main(String a[]) {
        Solution obj = new Solution();
        int[] input = new int[]{1, 2, 3, 4};
        int[] result = obj.runningSum(input);
        System.out.println(Arrays.toString(result));
    }
}
```

Task-2:Richest Customer Wealth

```
class Solution {
    public int maximumWealth(int[][] accounts) {
        int maxWealth = 0;
        for (int[] customer : accounts) {
            int currentWealth = 0;
            for (int bankBalance : customer) {
                currentWealth += bankBalance;
            }
            maxWealth = Math.max(maxWealth, currentWealth);
        }

        return maxWealth;
    }
}
```

SESSION-5

Task-1: Squares of a Sorted Array

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        for(int i=0;i<nums.length;i++){
            nums[i]=nums[i]*nums[i];
        }
        for(int i=0;i<nums.length-1;i++){
```

```
            for(int j=0;j<nums.length-i-1;j++){
                if(nums[j]>nums[j+1]){
                int temp=nums[j];
                nums[j]=nums[j+1];
                nums[j+1]=temp;}
            }
        }
        return nums;
    }
    public static void main(String a[]){
        Solution obj = new Solution();
        int[] result= obj.sortedSquares(new int[]{-4,-1,0,3,10});
         System.out.println(result);
    }
}
```

Task-2: Find Pivot Index

```
class Solution {
    public int pivotIndex(int[] nums) {
        int totalSum = 0;
        for (int num : nums) {
            totalSum += num;
        }
        int leftSum = 0;
        for (int i = 0; i < nums.length; i++) {
```

```
            int rightSum = totalSum - leftSum - nums[i];
            if (leftSum == rightSum) {
                return i;
            }
            leftSum += nums[i];
        }
        return -1;
    }
}
```